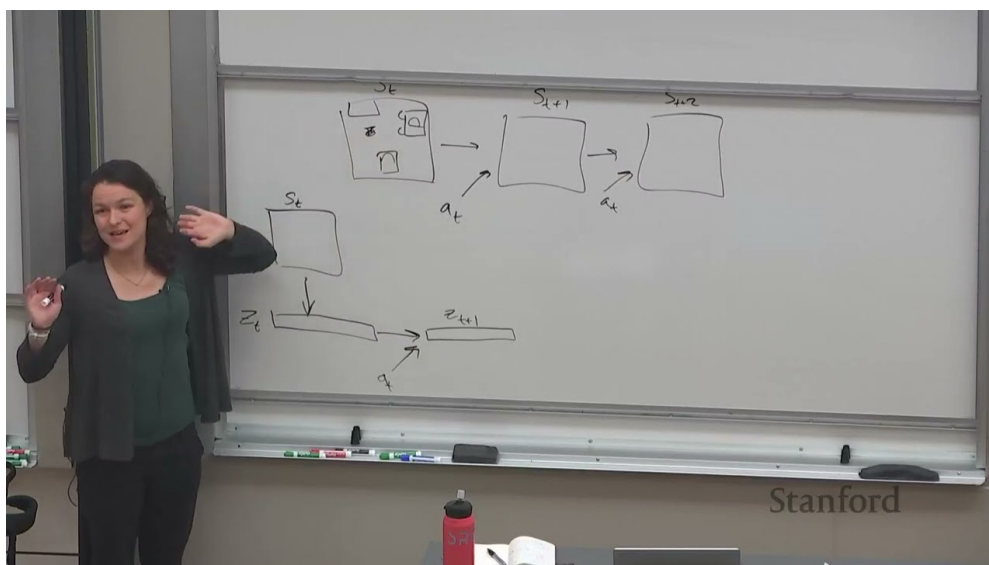


课程笔记

CS224R Lecture 11: 基于模型的强化学习

基于公开课程资料整理

2025 年春季



视频作者/频道: Stanford Online

发布日期: 2025

视频时长: 约 80 分钟

讲者: Chelsea Finn

课程主页: cs224r.stanford.edu

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1	课程回顾：算法分类总览	2
2	Model-Based RL 的核心思想	2
2.1	什么是 Model-Based RL	2
2.2	为什么要学习模型	3
2.3	本章小结	3
3	如何使用学到的模型	3
3.1	方法 1：基于模型的规划 (Planning)	3
3.1.1	随机打靶法 (Random Shooting)	4
3.1.2	CEM (Cross-Entropy Method)	4
3.1.3	方法 1a：基于梯度的规划	5
3.2	方法 2：使用模型生成合成数据	5
3.3	处理模型误差	6
3.4	本章小结	6
4	案例研究：灵巧操作中的 Model-Based RL	6
4.1	何时使用 Model-Based RL	7
4.2	本章小结	7
5	总结与延伸	7

1 课程回顾：算法分类总览

本讲开头，Chelsea Finn 对课程迄今为止讨论过的所有算法做了一个高层回顾。

算法分类框架

- **在线 (Online) vs 离线 (Offline)**: 在线算法允许在训练中持续与环境交互收集数据；离线算法仅使用给定的固定数据集。
- **Online 类别下**:
 - On-policy RL: 仅使用当前策略产生的数据 (如 REINFORCE, PPO)
 - Off-policy RL: 可以使用历史策略的数据 (如 SAC, TD3)
- **Offline 类别下**:
 - Imitation Learning: 行为克隆 (BC)、DAgger
 - Offline RL: CQL 等约束策略不偏离数据分布的方法

2 Model-Based RL 的核心思想

2.1 什么是 Model-Based RL

与 model-free 方法不同，model-based RL 显式地学习环境的**动力学模型** (dynamics model)，然后利用该模型来做决策。

动力学模型

学习一个函数 $f_\phi(\mathbf{s}, \mathbf{a})$ 来近似环境的状态转移 $p(\mathbf{s}'|\mathbf{s}, \mathbf{a})$:

$$f_\phi(\mathbf{s}, \mathbf{a}) \approx \mathbf{s}'$$

训练目标是最小化预测误差:

$$\min_{\phi} \sum_i \|f_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$$

其中数据 $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$ 来自与环境的交互。

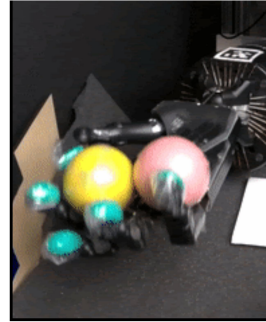
The plan for today

1. Model-based reinforcement learning
 - a. Key idea
 - b. How to learn a dynamics model? (in brief)
 - c. How to use a learned dynamics model?
 - i. Planning
 - ii. Data generation
2. Case study in dexterous robotic manipulation

Key learning goals:

- How to learn and use dynamics models
- The **key challenges** and trade-offs arising in model-based RL

How to get a robot to do this!



5

图 1: 本讲大纲: Model-Based RL 的核心内容¹

2.2 为什么要学习模型

Model-Based vs Model-Free 的核心权衡

- **数据效率**: Model-based 方法通常比 model-free 方法数据效率高得多。学到的模型可以用来“想象”无数种可能的轨迹，而不需要真实执行。
- **代价**: 模型不完美，基于不准确模型的决策可能导致性能下降。这就是**模型偏差** (model bias) 问题。

2.3 本章小结

Model-Based RL 的核心思路是“先学世界模型，再用模型做规划”。它在数据稀缺场景（如机器人）中特别有价值，但需要妥善处理模型误差。

3 如何使用学到的模型

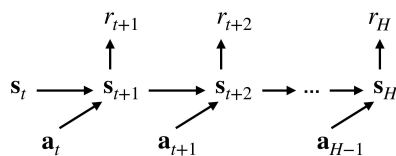
3.1 方法 1: 基于模型的规划 (Planning)

有了动力学模型后，可以直接在模型中搜索最优动作序列：

$$\max_{\mathbf{a}_{t:t+H}} \sum_t r(\mathbf{s}_t, \mathbf{a}_t)$$

¹Slides 第 5 页。

Approach 1: Optimize over actions using model $\max_{\mathbf{a}_{t:t+H}} \sum_t r(\mathbf{s}_t, \mathbf{a}_t)$ "planning"



Approach 1b:
via sampling
(i.e. gradient-free optimization)

Algorithm:

1. Run some policy (e.g. random policy) to collect data $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. Learn model $f_\phi(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. Iteratively sample action sequences, run through model $f_\phi(\mathbf{s}, \mathbf{a})$ to choose actions

15

图 2: 方法 1b: 通过采样进行无梯度优化的规划²

具体算法流程:

1. 运行某个策略 (如随机策略) 收集数据 $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. 学习模型 $f_\phi(\mathbf{s}, \mathbf{a})$
3. 迭代地采样动作序列, 通过模型前向推演选择最优动作

3.1.1 随机打靶法 (Random Shooting)

最简单的方法是随机采样 N 个动作序列, 在模型中 rollout 每个序列, 选择累积奖励最高的那个序列的第一个动作。

3.1.2 CEM (Cross-Entropy Method)

CEM 是一种更高级的无梯度优化方法:

1. 初始化动作分布 (如高斯分布)
2. 从分布中采样 N 个动作序列
3. 在模型中评估每个序列的累积奖励
4. 选择 top- K 的 "精英" 序列
5. 用精英序列更新动作分布的均值和方差
6. 重复若干轮迭代

MPC (Model Predictive Control)

在实际执行时, 通常采用 MPC 范式: 每一步只执行规划得到的第一个动作, 然后在新的状态上重新规划。这可以部分缓解模型在长时间跨度上的累积误差。

²Slides 第 15 页。

3.1.3 方法 1a: 基于梯度的规划

如果模型可微, 可以直接对动作序列求梯度来优化。但这种方法在实践中容易陷入局部最优, 且对模型的光滑性要求较高。

3.2 方法 2: 使用模型生成合成数据

Model-based policy optimization

Option 1: Distill planner's actions into a policy

(i.e. train policy to match actions taken by planner)

+ no longer compute intensive at test time

- still limited to short-horizon problems

How might we solve longer-horizon problems using a model?

1. Plan with terminal value function
2. Augment model-free RL methods with data from model

Let's focus on #2

25

图 3: Model-Based 策略优化的两种路线³

规划方法的主要局限:

- 测试时计算开销大 (每步都需要优化)
- 受限于短视野问题

解决长视野问题的两种思路:

1. **带终端值函数的规划**: 用短视野规划 + 学到的值函数作为终端估值
2. **Dyna 风格方法**: 用模型生成合成数据来增强 model-free RL 的训练

Dyna 算法

Dyna 是 model-based RL 的经典范式:

1. 收集真实数据, 训练/更新动力学模型
2. 用模型生成合成数据 (synthetic rollouts)
3. 将合成数据与真实数据混合, 用 model-free RL 算法更新策略

核心优势是利用模型的泛化能力将有限的真实数据”放大”为大量训练数据。

³Slides 第 25 页。

3.3 处理模型误差

模型误差的累积效应

模型在每一步的小误差会沿时间轴累积。如果规划视野为 H ，且每步误差为 ϵ ，则 H 步后的总误差可能高达 $O(H \cdot \epsilon)$ 甚至更大（如果动力学是不稳定的）。因此，盲目信任模型会导致策略“利用”模型的缺陷（model exploitation）。

应对策略：

- **模型集成 (Ensemble)**：训练多个模型，用不同模型的预测来估计不确定性
- **短视野 rollout**：限制模型 rollout 的步数
- **持续更新模型**：随着收集新数据不断更新模型

3.4 本章小结

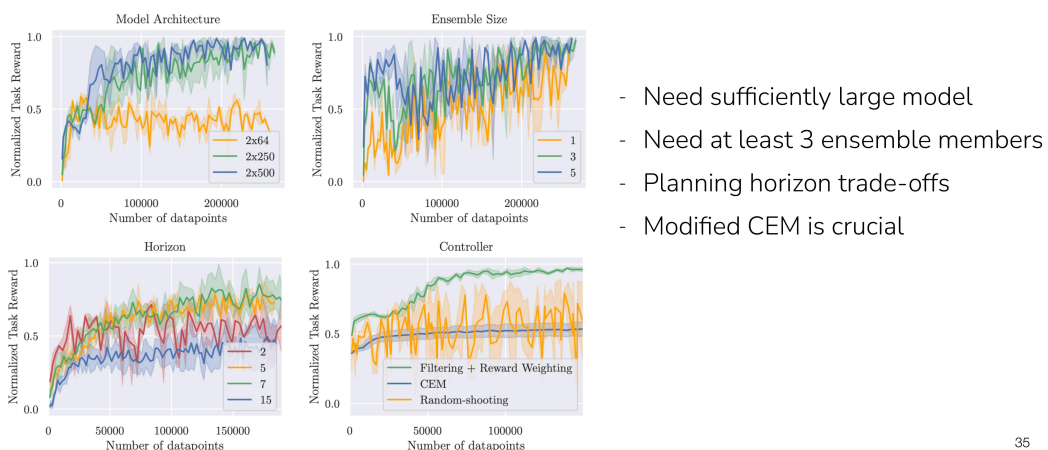
模型可以通过两种方式使用：(1) 直接规划（优化动作序列），(2) 生成合成数据增强 model-free RL。两种方式都需要妥善处理模型误差。

4 案例研究：灵巧操作中的 Model-Based RL

讲者介绍了一个具体案例，使用 model-based RL 让机器人灵巧手进行物体操作。

Case study: Model-based RL for dexterous manipulation

Simulated ablations



35

图 4: 灵巧操作的模拟消融实验结果⁴

关键实验发现：

- 需要足够大的模型架构（2x250 或 2x500 隐藏单元）
- 至少需要 3 个集成成员来获得可靠的不确定性估计

⁴Slides 第 35 页。

- 规划视野存在权衡：太短看不到足够远的未来，太长模型误差累积
- 改进的 CEM 控制器（如 Filtering + Reward Weighting）比 Random Shooting 好得多

4.1 何时使用 Model-Based RL

Model-Based RL 的适用场景

- 数据收集代价高昂（如真实机器人）
- 状态空间相对低维且可预测
- 需要快速适应新任务（模型可迁移）
- 动力学具有一定的可学习结构

不适合的场景：高维观测（如图像）且动力学高度随机的环境。

4.2 本章小结

灵巧操作案例展示了 model-based RL 在数据效率上的优势，以及集成、规划视野等超参数对性能的关键影响。

5 总结与延伸

Model-Based RL 是 RL 工具箱中的重要组成部分，其核心优势在于数据效率。

关键点：

1. 学习动力学模型可以大幅减少所需的真实交互数据
2. 模型可用于 planning（CEM/MPC）或生成合成数据（Dyna 风格）
3. 模型误差是核心挑战，需要通过集成、短视野 rollout、持续更新来缓解
4. 在机器人操作等数据昂贵的场景中特别有价值

拓展阅读

- Nagabandi et al., “Neural Network Dynamics for Model-Based Deep RL with Model-Free Fine-Tuning,” ICRA 2018
- Chua et al., “Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models (PETS),” NeurIPS 2018
- Janner et al., “When to Trust Your Model: Model-Based Policy Optimization (MBPO),” NeurIPS 2019
- Sutton, “Dyna, an Integrated Architecture for Learning, Planning, and Reacting,” 1991